

Paddle静态图执行流程

目录

- 概述
- 1 整体结构
- 2 静态图组网DSL
 - 2.1 基础组网数据结构
 - 2.2 具体组网流程描述
 - 2.3 具体示例
- 3 静态图执行器
 - 3.1 执行器相关数据结构
 - 3.2 执行器关键流程
 - 3.3 核心逻辑详解
 - 3.3.1 Kernel选择
- 4 Operator与Kernel管理
 - 4.1 Fluid Operator管理
 - 4.1 Phi Kernel管理

概述

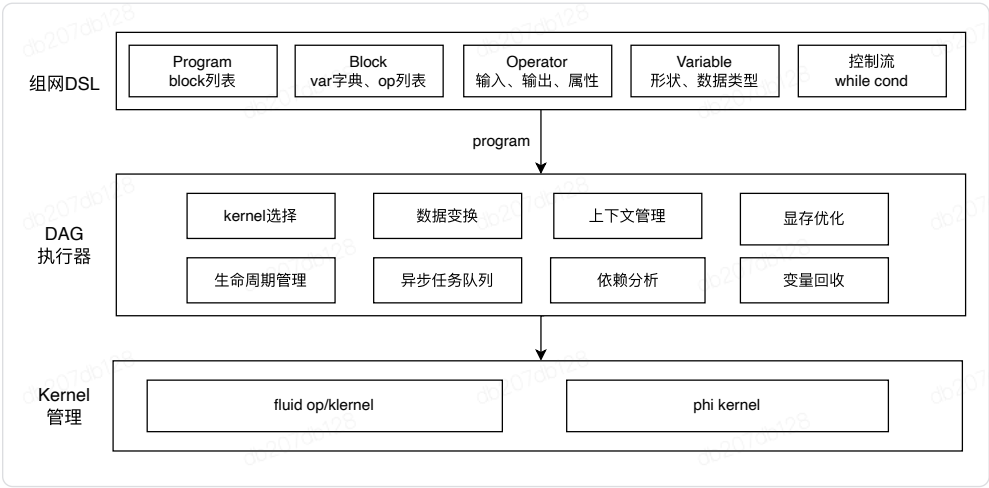
本文介绍Paddle静态图下组网与执行流程，出于简单考虑，主要包括

- 新执行器
- 单卡训练

暂不包括

- 编译器
- 分布式
- 动转静

1 整体结构



2 静态图组网DSL

2.1 基础组网数据结构

Program: python层描述计算图结构，一个program包含多个block

```
</> python/paddle/fluid/framework.py Python | 收起 ^

1 def __init__(self):
2     self.desc = core.ProgramDesc()
3     self.blocks = [Block(self, 0)]
```

```
4 self.current_block_idx = 0
```

Block: python层描述一个block结构, block包含Operator列表, 与Variable字典

```
</> python/paddle/fluid/framework.py Python | 收起 ^

1 def __init__(self, program, idx):
2     self.desc = program.desc.block(idx)
3     self.vars = collections.OrderedDict() # var_name --> var
4     self.ops = list() # operator list
5     self.program = program
6     self.removed_vars = collections.OrderedDict()
```

Operator: python层描述op的结构, 主要包括类型、输入、输出、属性

```
</> python/paddle/fluid/framework.py Python | 收起 ^

1 def __init__(self,
2     block,
3     desc,
4     type=None,
5     inputs=None,
6     outputs=None
7     attrs=None):
```

Variable: python层描述一个变量的结构, 包括名称、类型、形状、数据类型

```
</> python/paddle/fluid/framework.py Python | 收起 ^

1 def __init__(self,
2     block,
3     type=core.VarDesc.VarType.LOD_TENSOR,
4     name=None,
5     shape=None,
6     dtype=None,
7     ...
8     stop_gradient=False,
9     ...
10    **kwargs):
```

ProgramDesc: C++层描述Program数据结构, 包含block列表, proto::ProgramDesc

```
</> paddle/fluid/framework/program_desc.h C++ | 收起 ^

1 class ProgramDesc {
2     ...
3 private:
4     void InitFromProto();
5     proto::ProgramDesc desc_;
6     std::vector<std::unique_ptr<BlockDesc>> blocks_;
7     std::string cached_hash_str_;
```

BlockDesc: C++层描述Block数据结构, 包含op列表, var列表

```
</> C++ | 收起 ^

1 class BlockDesc {
2     ...
3 private:
4     ProgramDesc *prog_; // not_own
5     proto::BlockDesc *desc_; // not_own
6     bool need_update_; // block itself need_update, not aware of its ops_ and
7                         // vars_
8     std::deque<std::unique_ptr<OpDesc>> ops_;
9     std::map<std::string, std::unique_ptr<VarDesc>> vars_;
10    ...
11 };
```

OpDesc: C++层描述OP的数据结构

</> paddle/fluid/framework/op_desc.h

C++ | 收起 ^

```

1 class OpDesc {
2   ...
3 private:
4   friend class ProgramDesc;
5   ...
6   // it it really needed? or just mantain a ptr from block?
7   proto::OpDesc desc_;
8   BlockDesc *block_{nullptr}; // not_own
9   // input arg name => input variable names
10  VariableNameMap inputs_;
11  // output arg name => output variable names
12  VariableNameMap outputs_;
13  // attribute name => all original attrs
14  AttributeMap attrs_;
15  // runtime_attrs_ contains the attributes which used for dispatching kernel
16  // (use_mkldnn, use_cudnn, ...) or passing additional configuration for
17  // special heterogeneous kernel (workspace_size_MB, ...).
18  // The attributes in runtime_attrs_ are setted by framework (such as PASS),
19  // and not in the python api.
20  AttributeMap runtime_attrs_;
21  ...
22 };

```

VarDesc: c++层描述Variable, 主要包含proto::VarDesc

</> paddle/fluid/framework/var_desc.h

C++ | 收起 ^

```

1 class VarDesc {
2 private:
3   ...
4   proto::VarDesc desc_;
5   ...
6   uint64_t id_ = GenerateId();
7   uint64_t original_id_ = id_;
8   ...
9 };

```

Proto

</> paddle/fluid/framework/framework.proto

C++ | 收起 ^

```

1 ...
2 // OpProto describes a C++ framework::OperatorBase derived class.
3 message OpProto {
4   message VarDesc {
5     message Attr {
6       required string name = 1;
7       required AttrType type = 2;
8       optional int32 i = 3;
9       optional string s = 4;
10      repeated int32 ints = 5;
11    };
12
13    required string name = 1;
14    required VarType type = 2;
15    optional bool persistable = 3 [ default = false ];
16    // True if the variable is an input data and
17    // have to check the feed data shape and dtype
18    optional bool need_check_feed = 4 [ default = false ];
19    optional bool is_parameter = 5 [ default = false ];
20    optional bool stop_gradient = 6 [ default = false ];
21    repeated Attr attrs = 7;
22  }
23
24 message BlockDesc {

```

```
25 required int32 idx = 1;
26 required int32 parent_idx = 2;
27 repeated VarDesc vars = 3;
28 repeated OpDesc ops = 4;
29 ...
30 }
31 ...
32 message ProgramDesc {
33   reserved 2, 3; // For backward compatibility.
34   repeated BlockDesc blocks = 1;
35   optional Version version = 4;
36   optional OpVersionMap op_version_map = 5;
37 }
```

2.2 具体组网流程描述

1	阶段	关键流程	核心代码
2	组网阶段	打开静态图开关 paddle.enable_static()	paddle.enable_static 是 paddle.fluid.dygraph.base.disable_dygraph 别名, _functional_dygraph_context_manager 是动态图执行上下文管理器, disable_dygraph 调用上下文管理器 exit 式 <pre></> paddle/fluid/dygraph/base.py Py 1 _functional_dygraph_context_manager.__exit__(*sys.exc_info()</pre> 动态图模式下 _functional_dygraph_context_manager 会通过guard进行创建 <pre></> paddle/fluid/dygraph/base.py Py 1 global _functional_dygraph_context_manager 2 if _functional_dygraph_context_manager is None: 3 _functional_dygraph_context_manager = guard(4 place=_get_paddle_place(place)) 5 _functional_dygraph_context_manager.__enter__()</pre> # call disable_dygraph when Python exit <pre>CleanupFuncRegistrar.register(disable_dygraph)</pre> _functional_dygraph_context_manager 主要代码 <pre></> Py 1 with framework.program_guard(train, startup): 2 with framework.unique_name.guard(): 3 with framework._dygraph_guard(tracer): 4 with framework._dygraph_place_guard(expected_place): 5 yield</pre>
		创建Program sp = paddle.static.Program() mp = paddle.static.Program()	通过Program初始化方法创建Program
3		在program_guard(mp, sp) 上下文中进行组网	program_guard将创建的mp,sp转为全局program, 退出上下文前转回 <pre></> python/paddle/fluid/framework.py Py 1 @signature_safe_contextmanager 2 def program_guard(main_program, startup_program=None): 3 ... 4 main_program = switch_main_program(main_program) 5 if startup_program is not None: 6 check_type(startup_program, 'startup_program', Program, 7 'paddle.static.program_guard') 8 # Tag the program __is_start_up as True</pre>

```
9 startup_program._is_start_up_program_ = True
10 startup_program = switch_startup_program(startup_program)
11 try:
12     yield
13 finally:
14     switch_main_program(main_program)
15     if startup_program is not None:
16         switch_startup_program(startup_program)
17
18 def switch_main_program(program):
19     global _main_program_
20     prev_program = _main_program_
21     _main_program_ = program
22     return prev_program
```

调用paddle.tanh进行前向计算

LayerHelper是操作Operator辅助工具，如append_op到main program，创建输出等

```
</> python/paddle/tensor/math.py Py
1 def tanh(x, name=None):
2     ...
3     helper = LayerHelper('tanh', **locals())
4     out = helper.create_variable_for_type_inference(x.dtype)
5     helper.append_op(type='tanh', inputs={'X': x}, outputs={'Out': out})
6
```

```
</> python/paddle/fluid/layer_helper.py Py
1 def append_op(self, *args, **kwargs):
2     return self.main_program.current_block().append_op(*args, **kwargs)
```

调用block的append_op方法添加op到当前block，此处创建了一个空的op_desc，注意c++端op_desc 输入、输出、属性等信息此

```
</> python/paddle/fluid/framework.py Py
1 def append_op(self, *args, **kwargs):
2     ...
3     op_desc = self.desc.append_op()
4     # NOTE(Aurelius84): In case of @to_static, all VarBase(s) should
5     # be converted into Variable(s) with same name and block location.
6     # This is ONE and ONLY logic of type transformation of dy2static.
7     inputs = kwargs.get("inputs", None)
8     outputs = kwargs.get("outputs", None)
9     with param_guard(inputs), param_guard(outputs):
10         op = Operator(block=self,
11                        desc=op_desc,
12                        type=kwargs.get("type", None),
13                        inputs=inputs,
14                        outputs=outputs,
15                        attrs=kwargs.get("attrs", None))
16         self.ops.append(op)
17     return op
```

在Operator的构造方法中初始化op desc相关信息

```
</> python/paddle/fluid/framework.py Py
1 def __init__(self,
2              block,
3              desc,
4              type=None,
5              inputs=None,
6              outputs=None,
7              attrs=None):
8     ...
9     self.desc.set_type(type)
10    proto = OpProtoHolder.instance().get_op_proto(type)
11    ...
```

```
12         self.desc.set_input(in_proto.name, in_arg_names)
13     else:
14         self.desc.set_input(in_proto.name, [])
15     ...
16         self.desc.set_output(out_proto.name, out_arg_names)
17     ...
18         self._update_desc_attr(attr_name, op_attrs[attr_name])
19     ...
20     self.desc.check_attrs()
21     if self._has_kernel(type):
22         self.desc.infer_var_type(self.block.desc)
23         self.desc.infer_shape(self.block.desc)
```

执行反向过程
paddle.static.gradients
(y,x)

反向主要执行过程为从后向前遍历前向计算图，添加反向节点

```
</> python/paddle/fluid/backward.py

1 def calc_gradient(targets, inputs, target_gradients=None, no_grad_set=None):
2     ...
3     op_path = _find_op_path(block, targets, inputs, block_no_grad_set,
4                             op_path_dict)
5     ...
6     _append_backward_ops_(block,
7                             op_path,
8                             targets,
9                             block,
10                            no_grad_dict,
11                            grad_to_var,
12                            input_grad_names_set=input_grad_names_set,
13                            op_path_dict=op_path_dict,
14                            rename_var_map=rename_var_map)
15     ...
16     _append_backward_vars_(block, fwd_op_num, grad_to_var, grad_info_map)
17     prog._sync_with_cpp()
18     ...
```

5

2.3 具体示例

```
</> 静态图下计算tanh导数的简单示例 Python | 收起 ^

1 import numpy as np
2 import paddle
3
4
5 paddle.enable_static()
6
7 sp = paddle.static.Program()
8 mp = paddle.static.Program()
9
10 with paddle.static.program_guard(mp, sp):
11     x = paddle.static.data(name='x', shape=(1,), dtype=paddle.float32)
12     x.stop_gradient = False
13     y = paddle.tanh(x)
14     g = paddle.static.gradients(y, x)
15
16 exe = paddle.static.Executor()
17 exe.run(sp)
18 y,g = exe.run(mp, feed={'x': np.ones((1,), dtype=np.float32)}, fetch_list=(y,g))
19
20 print(y,g)
```

```
1 { // block 0
2     var x : LOD_TENSOR.shape(1,).dtype(float32).stop_gradient(False)
3     var tanh_0.tmp_0 : LOD_TENSOR.shape(1,).dtype(float32).stop_gradient(False)
4     var tanh_0.tmp_0@GRAD : LOD_TENSOR.shape(1,).dtype(float32).stop_gradient(False)
5     var tanh_0.tmp_0_shape : LOD_TENSOR.shape(1,).dtype(int32).stop_gradient(False)
6     var x@GRAD : LOD_TENSOR.shape(1,).dtype(float32).stop_gradient(False)
7
8     {Out=['tanh_0.tmp_0']} = tanh(inputs={X=['x']}, op_device = , op_namescope = /, op_role = 0, op_role_var = [],
with_quant_attr = False)
9     {Out=['tanh_0.tmp_0_shape']} = shape(inputs={Input=['tanh_0.tmp_0']}, op_device = , op_namescope = , op_role = 1,
op_role_var = [], with_quant_attr = False)
10    {Out=['tanh_0.tmp_0@GRAD']} = fill_constant(inputs={ShapeTensor=['tanh_0.tmp_0_shape']}, dtype = 5, force_cpu = False,
op_device = , op_namescope = , op_role = 1, op_role_var = [], place_type = -1, shape = [1], str_value = , use_mkldnn =
False, value = 1.0, with_quant_attr = False)
11    {X@GRAD=['x@GRAD']} = tanh_grad(inputs={Out=['tanh_0.tmp_0'], Out@GRAD=['tanh_0.tmp_0@GRAD']}, op_device = ,
op_namescope = /, op_role = 1, op_role_var = [], with_quant_attr = False)
12 }
```

3 静态图执行器

3.1 执行器相关数据结构

StandaloneExecutor 单卡执行器

</> paddle/fluid/framework/new_executor/standalone_executor.hC++ | 收起 ^

```
1 class StandaloneExecutor {
2 public:
3     ...
4     paddle::framework::FetchList Run(Scope* scope,
5                                     const std::vector<std::string>& feed_na
6                                     const std::vector<std::string>& fetch_n
7     ...
8 private:
9     platform::Place place_;
10    const ProgramDesc& prog_;
11    std::unordered_map<std::string, std::shared_ptr<InterpreterCore>>
12        interpretercores_;
13 };
14
```

InterpreterCore 静态图新执行器核心数据结构，包括图分析、kernel选择，多线程执行、GC等核心功能

</> paddle/fluid/framework/new_executor/interpretercore.hC++ | 收起 ^

```
1 class InterpreterCore {
2 public:
3     InterpreterCore(const platform::Place& place,
4                     const BlockDesc& block,
5                     const std::set<std::string>& skip_gc_vars,
6                     Scope* scope,
7                     bool used_for_jit = false);
8
9     ~InterpreterCore();
10    ...
11    paddle::framework::FetchList Run(
12        const std::vector<std::string>& feed_names,
13        const std::vector<framework::LoDTensor>& feed_tensors);
14
15    paddle::framework::FetchList Run(const std::vector<std::string>& feed_names);
16    ...
17 private:
18    bool is_build_;
```

```
19
20 platform::Place place_;
21 const BlockDesc& block_; // not owned
22 std::set<std::string> skip_gc_vars_;
23
24 interpreter::DependencyBuilder dependency_builder_;
25 ...
26 // from variable scope
27 std::vector<Variable*> var_list_;
28 std::map<std::string, int> name2id_;
29 std::vector<VariableMetaInfo> vec_meta_info_;
30
31 std::vector<Instruction> vec_instruction_; // deconstruct before OpFuncNode
32
33 // last_live_ops_[i] contains the id of operatos that last access var[i]
34 std::map<size_t, std::set<size_t>> last_live_ops_;
35
36 std::vector<size_t> dependency_count_;
37 std::atomic<size_t> unfinished_op_numer_{0};
38 ...
39 std::shared_ptr<interpreter::AsyncWorkQueue> async_work_queue_;
40 ...
41 std::unique_ptr<InterpreterCoreGarbageCollector> gc_;
42
43 std::future<std::unique_ptr<AtomicVectorSizeT>> atomic_deps_;
44 std::future<std::unique_ptr<AtomicVectorSizeT>> atomic_var_ref_;
45
46 bool used_for_jit_{false};
47 };
48
```

OpFuncNode

</> paddle/fluid/framework/new_executor/new_executor_defs.h C++ | 收起 ^

```
1 struct OpFuncNode {
2     // TODO(zhiqiu): Better make it unique_ptr
3     std::shared_ptr<OperatorBase> operator_base_;
4     std::map<std::string, std::vector<int>>> input_index;
5     std::map<std::string, std::vector<int>>> output_index;
6     ...
7     OpKernelComputeFunc kernel_func_;
8     platform::DeviceContext* dev_ctx_; // not owned
9
10    // fit for phi kernel
11    phi::Kernel* phi_kernel_{nullptr}; // not owned
12
13    OpFuncType type_;
14 };
```

Instruction

</> paddle/fluid/framework/new_executor/new_executor_defs.h C++ | 收起 ^

```
1 class Instruction {
2 public:
3     Instruction(size_t id,
4                 OpFuncNode&& op_func_node,
5                 const platform::DeviceContext& dev_ctx);
6 private:
7     size_t id_;
8     OpFuncNode op_func_node_;
9     ...
10    std::shared_ptr<RuntimeContext> runtime_ctx_;
11    std::shared_ptr<InterpreterCoreInferShapeContext> infershape_ctx_;
12    std::shared_ptr<ExecutionContext> execution_ctx_;
13
```



```
14 std::vector<size_t> gc_check_var_list_;
15 NextInstruction next_instruction_;
16
17 std::vector<EventInter> input_events_;
18 std::vector<EventInter> output_events_;
19
20 std::vector<std::pair<Variable*, Variable*>> vec_inplace_in_to_out_;
21 };
```

3.2 执行器关键流程

1	阶段	关键流程	核心代码
2	执行阶段	创建执行器 exe=paddle.staitc.Executor() 执行初始化程序 exe.run(sp)	</> paddle/fluid/executor.pyPython <pre>1 def __init__(self, place=None): 2 ... 3 # NOTE: Whether to use experimental executor `StandaloneExecutor`. 4 self._enable_interpreter_core = _is_enable_standalone_executor() 5 self._executor_cache = _ExecutorCache() 6 ...</pre>
3		执行主程序 y, = exe.run(mp, feed={'x': np.ones((1.), dtype=np.float32)}, fetch_list=(y,))	</> python/paddle/fluid/executor.pyC++ <pre>1 def _run_impl(self, program, feed, fetch_list, feed_var_name, 2 fetch_var_name, scope, return_numpy, use_program_cache, 3 return_merged, use_prune): 4 ... 5 program, new_exe = self._executor_cache.get_program_and_executor(6 program, feed, fetch_list, feed_var_name, fetch_var_name, 7 self.place, scope) 8 ... 9 return new_exe.run(scope, list(feed.keys()), fetch_list, 10 return_numpy) 11 ...</pre> 从LRU Cache中根据program,feed,fetch_list的哈希值查找program,exe </> python/paddle/fluid/executor.pyPython <pre>1 class _ExecutorCache(object): 2 class _CachedData(object): 3 def __init__(self, program, feed, fetch_list, feed_var_name, 4 fetch_var_name, place, scope): 5 ... 6 self.key = hash(7 _get_strong_program_cache_key_for_new_exe(8 self.program, feed, fetch_list)) 9 ... 10 def __hash__(self): 11 return self.key 12 13 def __init__(self): 14 # NOTE(Ruibiao): Wrap the lru_cache in constructor so that the cache is local to 15 # the _ExecutorCache instance, otherwise a global cache may not be released after 16 # the Executor instance deleted 17 self._get_cached_program_and_executor = lru_cache(maxsize=8)(</pre>

```

18         self._get_program_and_executor)
19
20     def get_program_and_executor(self, program, feed, fetch_list, feed_var_name,
21                                fetch_var_name, place, scope):
22         return self._get_cached_program_and_executor(
23             self._CachedData(program, feed, fetch_list, feed_var_name,
24                               fetch_var_name, place, scope))
25
26     def _get_program_and_executor(self, cached_data):
27         ...
28         program = _add_feed_fetch_ops(program=inner_program,
29                                       feed=feed,
30                                       fetch_list=fetch_list,
31                                       feed_var_name=feed_var_name,
32                                       fetch_var_name=fetch_var_name,
33                                       use_fetch_v2=True)
34         ...
35         new_exe = _StandaloneExecutor(place, new_program, scope)
36         return new_program, new_exe
37

```

调用_StandaloneExecutor创建执行器,

</> python/paddle/fluid/executor.py

Python

```

1 class _StandaloneExecutor(object):
2
3     def __init__(self, place, main_program, scope):
4         self._place = core.Place()
5         self._place.set_place(place)
6         self._main_program = main_program
7         self._scope = scope
8         self._new_exe = self._create_new_executor()
9
10    def _create_new_executor(self):
11        new_exe = core.StandaloneExecutor(self._place, self._main_program.desc)

```

调用_StandaloneExecutor run方法执行program

</> python/paddle/fluid/executor.py/_StandaloneExecutor

Python

```

1 def run(self, scope, feed_names, fetch_list, return_numpy=True):
2     fetch_list = self._check_fetch(fetch_list)
3     tensors = self._new_exe.run(scope, feed_names,
4                                 fetch_list)._move_to_list()
5
6     if return_numpy:
7         return as_numpy(tensors, copy=True)
8     else:
9         return tensors

```

执行StrandaloneExecutor::Run, 通过GetInterpreterCore方法获取InterpreterCore, 然后调用run方法执行

</> paddle/fluid/framework/new_executor/standalone_executor.cc

C++

```

1 paddle::framework::FetchList StandaloneExecutor::Run(
2     Scope* scope,
3     const std::vector<std::string>& feed_names,
4     const std::vector<std::string>& fetch_names) {
5     ...
6     auto core = GetInterpreterCore(scope, prog_, feed_names, fetch_names, false);
7     ...
8     return core->Run(feed_names);
9 }

```

GetInterpreterCore根据feed_names, fetch_names, scope 从interpretercores查找interpretercore, 查不到创建新的InterpreterCore, 并放入nterpretercores中, interpretercores是interpretercore的缓存

调用InterpreterCore的Run方法执行。第一次执行, 调用build_op_func_list, 按顺序执行OP, 包括op_func_nodes构造、Kernel选择、数据InferShape、Kernel执行等步骤。执行完之后调用Convert进行图优化、依赖分析等操作, 创建instruction vector, 以供下一次执行。

第二次执行会创建一个异步任务队列执行Instruction, 并创建GarbageCollector进行垃圾回收等操作

```
</> paddle/fluid/framework/new_executor/interpretercore.cc C++
1 paddle::framework::FetchList InterpreterCore::Run(
2     const std::vector<std::string>& feed_names) {
3     ...
4     if (!is_build_) {
5         ...
6         paddle::framework::interpreter::build_variable_scope(
7             block_, &var_scope_, create_local_scope_);
8
9         std::vector<paddle::framework::OpFuncNode> op_func_nodes;
10        paddle::framework::interpreter::build_op_func_list(place_,
11                                                            block_,
12                                                            skip_gc_vars_,
13                                                            &op_func_nodes,
14                                                            &var_scope_,
15                                                            create_local_scope_,
16                                                            used_for_jit_);
17        is_build_ = true;
18        SetFeedVarsInplaceSkip(feed_names);
19        // convert vec func_list to graph
20        Convert(&op_func_nodes);
21    } else {
22        // For the program that only run once, it is no need to
23        // create work_queue, so the async_work_queue_ is created
24        // until the second step run.
25        async_work_queue_ = GetWorkQueue();
26
27        // lazy initialization of gc, do not create gc is the program only run once
28        if (!gc_) {
29            gc_ = CreateInterpreterCoreGarbageCollector(place_, vec_instruction_);
30        }
31        ExecuteInstructionList(vec_instruction_);
32        ...
33    }
34    ...
35 }
```

build_op_func_list包括OpFuncNode构造、Kernel选择、数据转换、InferShape、执行Kernel、后置处理

```
</> paddle/fluid/framework/new_executor/interpretercore_util.cc C++
1 void build_op_func_list(const platform::Place& place,
2                         const framework::BlockDesc& block,
3                         const std::set<std::string>& skip_gc_vars,
4                         std::vector<OpFuncNode*> vec_func_list,
5                         VariableScope* var_scope,
6                         bool use_local_scope,
7                         bool used_for_jit) {
8     ...
9     // step 1: build OpFuncNode
10    OpFuncNode op_func_node;
11    op_func_node.operator_base_ = ops[i];
12    op_func_node.input_index = ins_name2id;
13    op_func_node.output_index = outs_name2id;
14
15    try {
16        if (dynamic_cast<framework::OperatorWithKernel*>(op) == nullptr) {
```

```

17 // op is not a operatorwithkernel, so directly run OperatorBase::Run()
18 deal_operator_base(
19     place, var_scope, ops[i], &op_func_node, local_scope);
20 } else {
21     auto op_with_kernel = const_cast<framework::OperatorWithKernel*>(
22         static_cast<const framework::OperatorWithKernel*>(op));
23     ...
24     RuntimeContext runtime_context({}, {});
25     runtime_context.inputs.swap(ins_map);
26     runtime_context.outputs.swap(outs_map);
27     Scope scope, *runtime_scope = &scope;
28     ...
29     auto& pool = platform::DeviceContextPool::Instance();
30     auto* dev_ctx = pool.Get(place);
31     VLOG(4) << "get dev_ctx";
32     auto exec_ctx = ExecutionContext(
33         *op_with_kernel, *runtime_scope, *dev_ctx, runtime_context);
34     VLOG(4) << "get exec_ctx";
35     auto expected_kernel_key =
36         op_with_kernel->GetExpectedKernelType(exec_ctx);
37     ...
38     // step 2. select op kernel
39     auto run_phi_kernel = false;
40     if (phi::KernelFactory::Instance().HasCompatiblePhiKernel(
41         op_with_kernel->Type())) {
42         auto phi_kernel_key = op_with_kernel->ChoosePhiKernel(exec_ctx);
43         auto phi_kernel_name = op_with_kernel->PhiKernelSignature()->name;
44
45         if (op_with_kernel->PhiKernel()->IsValid()) {
46             run_phi_kernel = true;
47         } else {
48             if (!op_with_kernel->SupportsKernelType(expected_kernel_key)) {
49                 auto phi_cpu_kernel_key = FallBackToCpu(
50                     expected_kernel_key, phi_kernel_key, *op_with_kernel);
51                 op_with_kernel->ResetPhiKernel(
52                     new phi::Kernel(phi::KernelFactory::Instance().SelectKernel(
53                         phi_kernel_name, phi_cpu_kernel_key)));
54                 if (op_with_kernel->PhiKernel()->IsValid()) {
55                     VLOG(6) << "Static mode PrepareImpl - kernel name: "
56                         << phi_kernel_name
57                         << " | kernel key: " << phi_cpu_kernel_key
58                         << " | kernel: " << *(op_with_kernel->PhiKernel());
59                     op_with_kernel->ResetKernelType(new OpKernelType(
60                         TransPhiKernelKeyToOpKernelType(phi_cpu_kernel_key)));
61                     run_phi_kernel = true;
62                 }
63             }
64         }
65     }
66     VLOG(4) << "if run phi kernel? : " << run_phi_kernel;
67     if (!run_phi_kernel) {
68         op_with_kernel->ChooseKernel(exec_ctx);
69         op_func_node.kernel_func_ = *op_with_kernel->kernel_func();
70     } else {
71         op_func_node.phi_kernel_ = op_with_kernel->PhiKernel();
72     }
73     auto kernel_type = *(op_with_kernel->kernel_type());
74     if (kernel_type.place_ != dev_ctx->GetPlace()) {
75         dev_ctx = pool.Get(kernel_type.place_);
76     }
77     op_func_node.dev_ctx_ = dev_ctx;
78     ...
79     // step 3. data transform
80     VariableValueMap& ins_map_temp = runtime_context.inputs;
81     VariableValueMap& outs_map_temp = runtime_context.outputs;

```

```

82     ApplyDataTransform(kernel_type,
83                         place,
84                         &ins_map_temp,
85                         &outs_map_temp,
86                         var_scope,
87                         &op_func_node,
88                         vec_func_list,
89                         use_local_scope);
90     VLOG(4) << "apply data transform done. ";
91
92     // step 4. infershape, see OperatorWithKernel::RunImpl in operator.cc
93     // for why.
94     if (!(!op->HasAttr(kAllKernelsMustComputeRuntimeShape) &&
95             op->Attr<bool>(kAllKernelsMustComputeRuntimeShape))) {
96         InterpreterCoreInferShapeContext infer_shape_ctx(*op,
97                                                         runtime_context);
98         // TODO(Aurelius84): In case of control flow ops, they are NOT
99         // inherited from OperatorWithKernel.
100        op_with_kernel->Info().infer_shape(&infer_shape_ctx);
101    }
102
103    // step 5. run kernel
104    if (run_phi_kernel) {
105        VLOG(1) << "start run phi kernel. ";
106        phi::KernelContext phi_kernel_context;
107        op_with_kernel->BuildPhiKernelContext(
108            runtime_context, dev_ctx, &phi_kernel_context);
109        (*op_func_node.phi_kernel_)(&phi_kernel_context);
110        VLOG(1) << "end run phi kernel. ";
111    } else {
112        VLOG(4) << "start run kernel. ";
113        // the place of exec_ctx maybe has changed.
114        op_func_node.kernel_func_(ExecutionContext(
115            *op_with_kernel, *runtime_scope, *dev_ctx, runtime_context));
116        VLOG(4) << "end run kernel. ";
117    }
118    ...
119 }
120
121 调用Convert, 创建vec_instruction_, 构建图依赖, 并计算last_live_ops进行显存优化
122 void InterpreterCore::Convert(
123     std::vector<paddle::framework::OpFuncNode>* op_func_nodes) {
124     ...
125     auto nodes = *op_func_nodes;
126
127     auto op_nums = nodes.size();
128     vec_instruction_.reserve(op_nums);
129     for (size_t op_idx = 0; op_idx < op_nums; ++op_idx) {
130         auto& op_func_node = nodes[op_idx];
131         auto* dev_ctx_ = stream_analyzer_.ParseDeviceContext(op_func_node);
132         vec_instruction_.emplace_back(op_idx, std::move(op_func_node), *dev_ctx_);
133     }
134     BuildOperatorDependences();
135     // calculate last_live_ops_
136     for (size_t op_idx = 0; op_idx < op_nums; ++op_idx) {
137         auto& instr = vec_instruction_[op_idx];
138         OpInOutInfo info;
139         std::set<size_t> gc_check_inputs;
140         for (auto& item : instr.Inputs()) {
141             for (auto id : item.second) {
142                 if (id == kEmptyVarIndex) {
143                     continue;
144                 }
145                 input_var2op_info_.at(id).push_back(op_idx);
146                 // var can be gc-ed

```

```

147         if (!info.IsBuilt()) {
148             info.Build(instr.OpBase());
149         }
150         auto* var_desc = var_scope_.VarDesc(id);
151         if (var_desc) {
152             if (info.IsInArgBufferNeeded(var_desc->Name())) {
153                 gc_check_inputs.insert(id);
154             }
155             } else {
156                 gc_check_inputs.insert(id);
157             }
158         }
159     }
160     ...
161     // clear the last_live_ops list for all vars in skip_gc_vars
162     for (const std::string& skip_gc_var : skip_gc_vars_) {
163         int var_id = var_scope_.GetIdByName(skip_gc_var);
164         if (var_id != -1) {
165             last_live_ops_[var_id].clear();
166             VLOG(8) << "Skip gc for var: " << skip_gc_var;
167         }
168     }
169     // shrink, find the downstream op that has no other op in the
170     // downstream list happens before it
171     // For example,
172     // b = op1(a)
173     // c = op2(a, b)
174     // in this case, a is the input of op1 and op2, we only need to check
175     // a after op2, because op2 always uses a after op1.
176     for (size_t i = 0; i < last_live_ops_.size(); ++i) {
177         std::set<size_t> minumum_last_live_ops;
178         for (size_t item : last_live_ops_[i]) {
179             bool not_before_any = true;
180             // find the op that is not executed before any
181             for (size_t other_item : last_live_ops_[i]) {
182                 if (dependency_builder_.OpHappensBefore(item, other_item)) {
183                     VLOG(8) << "happens_before: " << item << "->" << other_item
184                     << ", so skip " << item;
185                     not_before_any = false;
186                     break;
187                 }
188             }
189             if (not_before_any) {
190                 VLOG(8) << "last live op of var " << i << " "
191                 << var_scope_.GetNameById(i) << " : " << item << " "
192                 << vec_instruction_[item].OpBase()->Type();
193                 minumum_last_live_ops.insert(item);
194                 vec_instruction_[item].AddGCCheckVar(i);
195             }
196         }
197         last_live_ops_[i] = minumum_last_live_ops;
198         vec_meta_info[i].var_ref_count_ = last_live_ops_[i].size();
199     }
200     ...
201     for (size_t i = 0; i < vec_instruction_.size(); ++i) {
202         BuildAndCacheInstructionCtx(&vec_instruction_[i]);
203     }
204 }
205
206 调用BuildOperatorDependences构建图依赖
207 void InterpreterCore::BuildOperatorDependences() {
208     // analysis the dependences between ops, set the dependency_count_ and Call
209     // Schedule
210     auto op_nums = vec_instruction_.size();
211     dependency_count_.resize(op_nums);

```

```
212 auto op2downstream = dependency_builder_.Build(vec_instruction_);
213 for (size_t op = 0; op < vec_instruction_.size(); ++op) {
214     auto op_list = op2downstream[op];
215     std::vector<size_t> downstream_vector(op_list.begin(), op_list.end());
216     stream_analyzer_.Schedule(downstream_vector, &vec_instruction_, op);
217
218     for (auto inst_id : op_list) {
219         dependency_count_[inst_id]++;
220     }
221 }
222 }
223
224
225
226
```

3.3 核心逻辑详解

3.3.1 Kernel选择

理想情况下，在给定计算图和设备之后，每个op要执行的kernel可以静态确定，减少调度开销。但是，在飞桨旧Op体系下，kernel的选择是一个“动态”的选择，每个Op有 GetExpectedKernelType函数来选择kernel，该函数中会用到运行时信息。此外，对于Op输入/输出是没有显示约束的，例如一个gpubkernel可能会输出cputensor。新执行器采用了“一轮动态调度+缓存”的方式实现半静态化。新执行器第一个batch会以for-loop方式动态选择kernel执行，并缓存下kernel、context和datatransfer信息，并利用这些信息完成其余的预分析工作

判断是否为OperatorWithKernel 如果为OperatorBase，直接调用OperatorBase::Run方法	</> C++ 收起 ^ 1 if (dynamic_cast<framework::OperatorWithKernel*>(op) == nullptr) { 2 // op is not a operatorwithkernel, so directly run OperatorBase::Run() 3 deal_operator_base(4 place, var_scope, ops[i], &op_func_node, local_scope); 5 VLOG(4) << "deal_operator_base"; 6 } else { 7
准备上下文信息	</> C++ 收起 ^ 1 auto op_with_kernel = const_cast<framework::OperatorWithKerne 2 static_cast<const framework::OperatorWithKernel*>(op)); 3 VLOG(4) << "get op_with_kernel"; 4 // construct RuntimeContext and analysis KernelType 5 RuntimeContext runtime_context({}, {}); 6 runtime_context.inputs.swap(ins_map); 7 runtime_context.outputs.swap(outs_map); 8 VLOG(4) << "get RuntimeContext"; 9 Scope scope, *runtime_scope = &scope; 10 // NOTE(Ruibiao): We do not encourage directly using scope in 11 // But some OPs do have such behavior (e.g., cinn_launch OP). 12 // special treatment for them. 13 if (op_with_kernel->Type() == "cinn_launch") { 14 VLOG(6) << "OP(" << op_with_kernel->Type() 15 << ") use scope in kernel, " 16 "so pass a real scope to " 17 "ExecutionContext"; 18 runtime_scope = local_scope; 19 } 20 auto& pool = platform::DeviceContextPool::Instance(); 21 auto* dev_ctx = pool.Get(place); 22 VLOG(4) << "get dev_ctx"; 23 auto exec_ctx = ExecutionContext(24 *op_with_kernel, *runtime_scope, *dev_ctx, runtime_context 25 VLOG(4) << "get exec_ctx";


```
26 auto expected_kernel_key =
27     op_with_kernel->GetExpectedKernelType(exec_ctx);
28 VLOG(4) << "get expected_kernel_key";
29 // change device by the device_guard()
```

优先选择phi kernel

Phi kernel选择

</> C++ | 收起 ^

```
1 if (phi::KernelFactory::Instance().HasCompatiblePhiKernel(
2     op_with_kernel->Type())) {
3     auto phi_kernel_key = op_with_kernel->ChoosePhiKernel(exec_ctx);
4     auto phi_kernel_name = op_with_kernel->PhiKernelSignature()->name;
5     if (op_with_kernel->PhiKernel()->IsValid()) {
6         run_phi_kernel = true;
7     } else {
8         if (!op_with_kernel->SupportsKernelType(expected_kernel_key)) {
9             auto phi_cpu_kernel_key = FallBackToCpu(
10                 expected_kernel_key, phi_kernel_key, *op_with_kernel);
11             op_with_kernel->ResetPhiKernel(
12                 new phi::Kernel(phi::KernelFactory::Instance().SelectKernel(
13                     phi_kernel_name, phi_cpu_kernel_key)));
14             if (op_with_kernel->PhiKernel()->IsValid()) {
15                 VLOG(6) << "Static mode PrepareImpl - kernel name: "
16                     << phi_kernel_name
17                     << " | kernel key: " << phi_cpu_kernel_key
18                     << " | kernel: " << *(op_with_kernel->PhiKernel());
19                 op_with_kernel->ResetKernelType(new OpKernelType(
20                     TransPhiKernelKeyToOpKernelType(phi_cpu_kernel_key)));
21                 run_phi_kernel = true;
22             }
23         }
24     }
25 }
```

调用HasCompatiblePhiKnerel查找Phi kernel

</> C++ | 收起 ^

```
1 phi::KernelFactory::Instance().HasCompatiblePhiKernel(
2     op_with_kernel->Type())
```

HasCompatiblePhiKernel 从kernels_中查找,

</> C++ | 收起 ^

```
1 bool HasCompatiblePhiKernel(const std::string& op_type) const {
2     return kernels_.find(TransToPhiKernelName(op_type)) != kernels_.end();
3 }
4
5 KernelNameMap kernels_;
6 using KernelNameMap = paddle::flat_hash_map<std::string, KernelKeyMap>;
7 using KernelKeyMap = paddle::flat_hash_map<KernelKey, Kernel, KernelKey::Hash>;
8
9 const std::string& TransToPhiKernelName(const std::string& fluid_op_name) {
10     return OpUtilsMap::Instance().GetBaseKernelName(fluid_op_name);
11 }
12
13 const std::string& GetBaseKernelName(const std::string& op_type) const {
14     ...
15     auto it = base_kernel_name_map_.find(op_type);
16     if (it == base_kernel_name_map_.end()) {
17         return op_type;
18     } else {
19         return it->second;
20     }
```



```

20 }
21 }
22
23 paddle::flat_hash_map<std::string, std::string> base_kernel_name_map_;
24
25 #define PD_REGISTER_BASE_KERNEL_NAME(op_type, base_kernel
26 PD_STATIC_ASSERT_GLOBAL_NAMESPACE(
27     PD_REGISTER_base_kernel_name_ns_check_##op_type,
28     "PD_REGISTER_BASE_KERNEL_NAME must be called in glo
29 static const ::phi::BaseKernelNameRegistrar
30     __registrar_base_kernel_name_for_##op_type(#op_type
31     int TouchBaseKernelNameSymbol_##op_type() { return 0; }
32
33 在 *_sig.cc 中注册fluid op名与phi名称之间映射关系

```

如果找到，调用op_with_kernel->ChoosePhiKernel 选择具体可执行Phi kernel，注意，此时可能返回Empty kernel

```

</> C++ | 收起 ^
1 uto phi_kernel_key = op_with_kernel->ChoosePhiKernel(exec_ctx);
2 auto phi_kernel_name = op_with_kernel->PhiKernelSignature()->name;
3
4 phi::KernelKey OperatorWithKernel::ChoosePhiKernel(
5     const ExecutionContext& ctx) const {
6     kernel_signature_.reset(
7         new phi::KernelSignature(std::move(GetExpectedPhiKernelArgs(ctx))));
8     VLOG(6) << *kernel_signature_.get();
9
10    kernel_type_.reset(
11        new OpKernelType(std::move(InnerGetExpectedKernelType(ctx))));
12
13    auto phi_kernel_name = kernel_signature_->name;
14    auto phi_kernel_key = TransOpKernelTypeToPhiKernelKey(*kernel_type_.get());
15    phi_kernel_.reset(new phi::Kernel(phi::KernelFactory::Instance().SelectKernel(
16        phi_kernel_name, phi_kernel_key)));
17    ...
18    return phi_kernel_key;
19 }
20
21 如果 op_with_kernel->PhiKernel()->IsValid() False 且 Fluid找不到对应Kernel，尝试fall back到
    phi cpu Kernel
22 if (op_with_kernel->PhiKernel()->IsValid()) {
23     run_phi_kernel = true;
24 } else {
25     if (!op_with_kernel->SupportsKernelType(expected_kernel_key)) {
26         auto phi_cpu_kernel_key = FallBackToCpu(
27             expected_kernel_key, phi_kernel_key, *op_with_kernel);
28         op_with_kernel->ResetPhiKernel(
29             new phi::Kernel(phi::KernelFactory::Instance().SelectKernel(
30                 phi_kernel_name, phi_cpu_kernel_key)));
31         if (op_with_kernel->PhiKernel()->IsValid()) {
32             VLOG(6) << "Static mode PrepareImpl - kernel name: "
33                 << phi_kernel_name
34                 << " | kernel key: " << phi_cpu_kernel_key
35                 << " | kernel: " << *(op_with_kernel->PhiKernel());
36             op_with_kernel->ResetKernelType(new OpKernelType(
37                 TransPhiKernelKeyToOpKernelType(phi_cpu_kernel_key)));
38             run_phi_kernel = true;
39         }
40     }
41 }
42
43 if (!run_phi_kernel) {
44     op_with_kernel->ChooseKernel(exec_ctx);
45     op_func_node.kernel_func_ = *op_with_kernel->kernel_func();

```

```
46 } else {
47     op_func_node.phi_kernel_ = op_with_kernel->PhiKernel();
48 }
```

4 Operator与Kernel管理

4.1 Fluid Operator管理

OperatorBase

</> paddle/fluid/framework/operator.h

C++ | 收起 ^

```
1 class OperatorBase {
2 public:
3     OperatorBase(const std::string& type,
4                 const VariableNameMap& inputs,
5                 const VariableNameMap& outputs,
6                 const AttributeMap& attrs);
7
8     virtual ~OperatorBase() {}
9
10    /// Executor will call this interface function to Run an op.
11    // The implementation should be written at RunImpl
12    void Run(const Scope& scope, const platform::Place& place);
13    ...
14 protected:
15     std::string type_;
16     // NOTE: in case of OpGrad, inputs_ contains:
17     // I (Inputs)
18     // O (Outputs)
19     // OG (Output Gradients)
20     VariableNameMap inputs_;
21
22     // NOTE: in case of OpGrad, outputs_ contains
23     // IG (Inputs Gradients)
24     VariableNameMap outputs_;
25     AttributeMap attrs_;
26     // NOTE: runtime_attrs_ contains the attributes which used for dispatching
27     // kernel (use_mkldnn, use_cudnn, ...) or passing additional configuration
28     // for special heterogeneous kernel (workspace_size_MB, ...).
29     // The attributes in runtime_attrs_ are setted by framework (such as PASS),
30     // and not in the python api.
31     AttributeMap runtime_attrs_;
32
33     // OpInfo
34     const OpInfo* info_;
35 };
```

OperatorWithKernel

</> paddle/fluid/framework/opeartor.h

C++ | 收起 ^

```
1 class OperatorWithKernel : public OperatorBase {
2 public:
3     using OpKernelFunc = std::function<void(const ExecutionContext&)>;
4     using OpKernelMap =
5         std::unordered_map<OpKernelType, OpKernelFunc, OpKernelType::Hash>;
6     ...
7     static paddle::flat_hash_map<std::string /* op_type */, OpKernelMap>&
8     AllOpKernels() {
9         static paddle::flat_hash_map<std::string, OpKernelMap> g_all_op_kernels;
10        return g_all_op_kernels;
11    }
12
13    virtual void InferShape(InferShapeContext* ctx) const;
```

```

14
15 void RuntimeInferShape(const Scope& scope,
16                        const platform::Place& place,
17                        const RuntimeContext& ctx) const override;
18 ...
19 virtual OpKernelType GetExpectedKernelType(const ExecutionContext& ctx) const;
20 ....
21 /* member functions for adapting to phi lib */
22 /** In the Tensor calculation library, the new Kernel adopts a clearer and
23  * more streamlined design. The arguments of the Kernel and the input and
24  * output arguments registered in the original OpMaker do not match in some
25  * cases, so we use map to record the arguments required by the kernel.
26  * When selecting Kernel during Op execution, select the arguments of the
27  * original Op according to the GetExpectedPhiKernelArgs returned arguments.
28  */
29 phi::KernelSignature GetExpectedPhiKernelArgs(
30     const ExecutionContext& ctx) const;
31
32 /* member functions for adapting to phi lib */
33 phi::KernelKey ChoosePhiKernel(const ExecutionContext& ctx) const;
34
35 void ChooseKernel(const ExecutionContext& ctx) const;
36
37 void BuildPhiKernelContext(const RuntimeContext& ctx,
38                           platform::DeviceContext* dev_ctx,
39                           phi::KernelContext* phi_kernel_context) const;
40
41 phi::KernelSignature* PhiKernelSignature() const {
42     return kernel_signature_.get();
43 }
44
45 phi::Kernel* PhiKernel() const { return phi_kernel_.get(); }
46
47 void ResetPhiKernel(phi::Kernel* kernel) const {
48     return phi_kernel_.reset(kernel);
49 }
50
51 const OpKernelType* kernel_type() const { return kernel_type_.get(); }
52 const OpKernelFunc* kernel_func() const { return kernel_func_.get(); }
53
54 void ResetKernelType(OpKernelType* kernel_type) {
55     kernel_type_.reset(kernel_type);
56 }
57 private:
58 ...
59 protected:
60 mutable std::unique_ptr<OpKernelType> kernel_type_;
61 mutable std::unique_ptr<OpKernelFunc> kernel_func_;
62 mutable std::unique_ptr<RuntimeContext> runtime_ctx_;
63 ...
64 mutable bool run_phi_kernel_ = false;
65 mutable bool run_kp_kernel_ = false;
66 mutable std::unique_ptr<phi::KernelSignature> kernel_signature_;
67 mutable std::unique_ptr<phi::Kernel> phi_kernel_;
68 mutable std::unique_ptr<phi::ArgumentMappingFn> arg_map_fn_;
69 };

```

OpKernelType

</> paddle/fluid/framework/op_kernel_type.h

C++ | 收起 ^

```

1 class OpKernelType {
2     ...
3     proto::VarType::Type data_type_;
4     DataLayout data_layout_;
5     platform::Place place_;

```

```
6 LibraryType library_type_;
7 int customized_type_value_;
8 };
```

4.1 Phi Kernel管理

KernelFactory

</> paddle/phi/core/kernel_factory.h

C++ | 收起 ^

```
1 class KernelFactory {
2 public:
3     static KernelFactory& Instance();
4
5     bool HasCompatiblePhiKernel(const std::string& op_type) const {
6         return kernels_.find(TransToPhiKernelName(op_type)) != kernels_.end();
7     }
8
9     KernelResult SelectKernelOrThrowError(const std::string& kernel_name,
10                                           const KernelKey& kernel_key,
11                                           bool use_gpubnn = false) const;
12
13     bool HasKernel(const std::string& kernel_name,
14                   const KernelKey& kernel_key) const;
15
16     const Kernel& SelectKernel(const std::string& kernel_name,
17                               const KernelKey& kernel_key) const;
18
19     KernelKeyMap SelectKernelMap(const std::string& kernel_name) const;
20
21     const KernelArgsDef& GetFirstKernelArgsDef(
22         const std::string& kernel_name) const;
23
24 private:
25     KernelFactory() = default;
26
27     KernelNameMap kernels_;
28 };
```

KernelKey

</>

C++ | 收起 ^

```
1 class KernelKey {
2 public:
3     KernelKey() = default;
4
5     KernelKey(Backend backend, DataLayout layout, Dat
6             : backend_(backend), layout_(layout), dtype_(
7
8     Backend backend() const { return backend_; }
9     DataLayout layout() const { return layout_; }
10    DataType dtype() const { return dtype_; }
11
12    struct Hash {
13        // Note: Now the number of bits we need does no
14        // no need to use 64 bits. If needed in the fut
15        // but now we don't over-design.
16        uint32_t operator()(const KernelKey& key) const
17    };
18    ...
19 private:
20    // In total should be smaller than 32.
21    constexpr static int kBackendBitLength = 8;
```

```
22 constexpr static int kDataLayoutBitLength = 4;
23 constexpr static int kDataTypeBitLength = 8;
24
25 Backend backend_{Backend::UNDEFINED};
26 DataLayout layout_{DataLayout::UNDEFINED};
27 DataType dtype_{DataType::UNDEFINED};
28 };
29
```

Kernel

</> paddle/phi/core/kernel_factory.hC++ | 收起 ^

```
1 class Kernel {
2 public:
3     ...
4     explicit Kernel(KernelFn fn, void* variadic_fn)
5         : fn_(fn), variadic_fn_(variadic_fn) {}
6
7     void operator()(KernelContext* ctx) const { fn_(ctx); }
8     ...
9     bool IsValid() const { return fn_ != nullptr; }
10
11 private:
12     KernelFn fn_{nullptr};
13     void* variadic_fn_ = nullptr;
14     KernelArgsDef args_def_;
15 };
16
17 using KernelFn = std::function<void(KernelContext* ctx)>;
```

KernelArgsDef

</> paddle/phi/core/kernel_factory.hC++ | 收起 ^

```
1 class KernelArgsDef {
2 public:
3     KernelArgsDef() = default;
4
5     void AppendInput(Backend backend,
6                     DataLayout layout,
7                     DataType dtype,
8                     std::type_index type_index) {
9         input_defs_.emplace_back(TensorArgDef(backend, layout, dtype, type_index));
10    }
11
12    void AppendOutput(Backend backend,
13                    DataLayout layout,
14                    DataType dtype,
15                    std::type_index type_index) {
16        output_defs_.emplace_back(TensorArgDef(backend, layout, dtype, type_index));
17    }
18
19    void AppendAttribute(AttributeType type_index) {
20        attribute_defs_.emplace_back(AttributeArgDef(type_index));
21    }
22    ...
23 private:
24     paddle::small_vector<TensorArgDef, kInputSmallVectorSize> input_defs_{{{}}};
25     paddle::small_vector<TensorArgDef, kOutputSmallVectorSize> output_defs_{{{}}};
26     paddle::small_vector<AttributeArgDef, kAttrSmallVectorSize> attribute_defs_{{{}}};
27 };
28
```

